Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Конструирование программного обеспечения»

Тема: «Создание ключевых функций»

Выполнил:

Студент 2 курса

Группы ПО-17

Бородин К. С.

Проверил:

Кулик А.Д.

Брест 2026

Цель работы: написать основную логику программы.

Ход работы

Основное задание: необходимо написать основную функцию для программы из предыдущих лабораторных работ. Ваша задача реализовать логику программы, которая будет выполняться при выборе пункта меню «Начать игру». Новые данные должны быть реализованы в виде классов.

Результатом данной лабораторной работы должно стать обновление программы, в котором появляется возможность выбрать главный пункт меню. Все настройки, сделанные в результате работы над предыдущими лабораторными работами, должны учитываться в логике функции этой лабораторной.

Листинг программы представлен в файле **lab3.cpp**.
Вариант: 1. Игра представляет собой спуск в подземелье и последовательные бои с противниками. Необходимо реализовать классы Character, Monster и Weapon. Реализовать функции экипировки оружия и выпадения его из монстров после победы. Бои должны быть поочередные (игрок выбирает действие, монстр просто наносит урон игроку).

Реализованные классы:

```
class Weapon {
public:
    std::string name;
    int damage;
    Weapon() : name("Кулаки"), damage(2) {}
    Weapon(std::string n, int d) : name(n), damage(d) {}
};

class Character {
public:
    std::string name;
    int maxHealth;
    int health;
    Weapon weapon;
    Character() : name("Герой"), maxHealth(10), health(10), weapon()
{}
    Character(std::string n, int h) : name(n), maxHealth(h),
health(h), weapon() {}
    void EquipWeapon(const Weapon& w) {
        weapon = w;
        std::cout << "Вы экипировали: " << w.name << " (урон: " <<
w.damage << ")\n";
    }
    int Attack() {
        return weapon.damage;
    }
    void TakeDamage(int dmg) {
        health -= dmg;
        if (health < 0) health = 0;
    }
    bool IsAlive() const {
```

```
        return health > 0;
    }
};

class Monster {
public:
    std::string name;
    int health;
    int damage;
    Weapon drop;
    bool hasDrop;
    Monster() : name("Слизень"), health(5), damage(1), drop(),
hasDrop(true) {}
    Monster(std::string n, int h, int d, Weapon w, bool hd)
        : name(n), health(h), damage(d), drop(w), hasDrop(hd) {
    }
    void TakeDamage(int dmg) {
        health -= dmg;
        if (health < 0) health = 0;
    }
    bool IsAlive() const {
        return health > 0;
    }
};
```

Функции экипировки оружия и его выпадение из монстров после победы:

```
void EquipWeapon(const Weapon& w) {
    weapon = w;
    std::cout << "Вы экипировали: " << w.name << " (урон: " <<
w.damage << ")\n";
}
std::cout << "\nИз монстра выпало оружие: " << monster.drop.name
    << " (урон: " << monster.drop.damage << ")\n";
std::cout << "1. Экипировать\n2. Оставить текущее\nВыбор: ";
int pick;
std::cin >> pick;
if (pick == 1) hero.EquipWeapon(monster.drop);
```

Реализация поочередных боев:

```
 while (hero.IsAlive() && monster.IsAlive()) {
     std::cout << "\nВаше HP: " << hero.health << "/" <<
hero.maxHealth;
     std::cout << " | Оружие: " << hero.weapon.name << " (урон: " <<
hero.weapon.damage << ")\n";
     std::cout << monster.name << " HP: " << monster.health << "\n";
     std::cout << "1. Атаковать\n2. Защищаться (пропустить ход)\n";
     std::cout << "Выберите действие: ";
     int act;
     std::cin >> act;
     bool defending = false;
     if (act == 1) {
         int dmg = hero.Attack();
         monster.TakeDamage(dmg);
```

```
            std::cout << "Вы нанесли " << dmg << " урона!\n";
        }
        else {
            defending = true;
            std::cout << "Вы защищаетесь.\n";
        }
        if (!monster.IsAlive()) {
            std::cout << monster.name << " побеждён!\n";
            break;
        }
        int incoming = monster.damage;
        if (defending) {
            incoming /= 2;
            std::cout << "Защита снизила урон вдвое. ";
        }
        hero.TakeDamage(incoming);
        std::cout << monster.name << " наносит вам " << incoming << "
урона!\n";
     }
```

Вывод: Написал основную логику программы посредством реализации новых классов и функций.